DEVELOPMENT OF AN AUTONOMOUS MARINE SOFTWARE USING ROS 2 AND GAZEBO

Project Report

Submitted by

DWARAPU SIREESHA

to

Rajiv Gandhi University of Knowledge Technologies, Srikakulam

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

Under the Guidance of

B. SURESH KUMAR

Scientist – F

Naval Science & Technological Laboratory (NSTL-DRDO)

Visakhapatnam

DECLARATION

I, Dwarapu Sireesha, hereby declare that the project report entitled “Development Of An Autonomous Marine Software Using Ros 2 And Gazebo” is a bonafide record of the work carried out by me during my internship/project work at Naval Science & Technological Laboratory (NSTL), Defence Research and Development Organisation (DRDO), Visakhapatnam, under the guidance of B. SURESH KUMAR, Scientist–F and Y.SREENIVAS RAO , Technology Director. The work presented in this report describes my implementation, integration, validation and analysis of a ROS 2–Gazebo autonomous marine software. Wherever the work or ideas of others have been referred to, the corresponding sources are acknowledged in the references.

I further declare that the experimental results, software architecture descriptions, controller parameters, simulation observations and limitations reported here have been presented to the best of my knowledge without fabrication. This report has not been submitted previously as the basis for the award of any other degree or diploma.

Place: Visakhapatnam

Date: _____

DWARAPU SIREESHA

B.Tech, Computer Science and Engineering

ACKNOWLEDGEMENT

I am pleased to present this project report on “Development Of An Autonomous Marine Software Using Ros 2 And Gazebo”. The work represents the development of an integrated autonomous marine research framework combining simulation, guidance, control, perception, mission autonomy, cooperative operation, safety supervision and operator visualization.

I express my sincere gratitude to my project guide, B. SURESH KUMAR , Scientist–F and

Y.SREENIVAS RAO , Technology Director, Naval Science & Technological Laboratory (NSTL-DRDO), for his guidance, technical feedback and encouragement throughout the development and validation of the system. His suggestions were valuable in shaping the work into a structured research framework rather than an isolated simulation or perception demonstration.

I am grateful to NSTL-DRDO, Visakhapatnam, for providing the opportunity to undertake this internship/project work in a professional defence research environment. The project gave me practical exposure to ROS 2 middleware, marine robotics, Gazebo simulation, guidance and control, computer vision, autonomous mission logic, multi-vehicle coordination and research-oriented verification.

I also thank Rajiv Gandhi University of Knowledge Technologies, Srikakulam and the Department of Computer Science and Engineering for academic support and for enabling me to undertake this work. Finally, I thank my family, friends and peers for their support during the implementation, testing and documentation phases.

DWARAPU SIREESHA

ABSTRACT

This project presents an integrated ROS 2-Gazebo framework for autonomous marine vehicle simulation, navigation, maritime target perception, mission execution, cooperative control, and operator supervision. The primary objective of the work is to develop a modular software architecture capable of autonomous path following, maritime target detection, target-oriented navigation, and coordinated unmanned surface vehicle operations in a simulated marine environment.

The framework combines spline-based reference-path generation and Line-of-Sight guidance with selectable PID and Sliding Mode Control strategies for autonomous navigation. A deep-learning-based vessel detection pipeline processes simulated camera imagery, while independent depth information is used for target-range estimation. Mission logic integrates systematic search, temporal detection confirmation, target acquisition, target-loss recovery, reacquisition, stand-off arrival, and inspection behavior. Cooperative leader-follower control is incorporated for coordinated vehicle operation, while the Marine Autonomy Control Station provides real-time telemetry, perception and mission monitoring, system-health assessment, and supervisory safety control.

The developed system demonstrates the successful integration of perception, guidance, control, mission management, cooperative autonomy, and operator supervision through ROS 2 communication. The framework was validated through staged subsystem testing and integrated simulation scenarios, providing a repeatable and extensible research platform for autonomous maritime operations. The completed architecture also establishes a foundation for future integration with physical sensors, real marine vehicles, hardware controllers, advanced environmental models, and expanded multi-agent capabilities.

TABLE OF CONTENTS

Chapter / Section	Page
DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CHAPTER 1 INTRODUCTION	1
1.1 Overview and Problem Statement	1
1.2 Objectives	1
1.3 Scope and Limitations	2
CHAPTER 2 LITERATURE SURVEY	3
2.1 ROS 2 for Autonomous Marine Systems	3
2.2 Marine Guidance and LOS Path Following	3
2.3 Feedback Control: PID and SMC	3
2.4 Deep-Learning Maritime Perception	4
2.5 Multi-USV Cooperative Control	4
2.6 Human Supervision and Safety	4
2.7 Literature Gap	4
CHAPTER 3 SYSTEM REQUIREMENTS	6

Chapter / Section	Page
CHAPTER 4 METHODOLOGY / SYSTEM DESIGN	8
CHAPTER 5 IMPLEMENTATION	14
CHAPTER 6 RESULTS AND DISCUSSION	17
CHAPTER 7 ADVANTAGES AND FUTURE EXTENDABILITY	27
CHAPTER 8 APPLICATIONS	28
CHAPTER 9 CONCLUSION	29
CHAPTER 10 FUTURE SCOPE	30
REFERENCES	32

CHAPTER 1

INTRODUCTION

1.1 Overview and Problem Statement

Autonomous surface vehicles are increasingly important for maritime surveillance, inspection, environmental monitoring, search operations and defence research. A practical autonomous marine platform must do more than move through a simulated environment: it must estimate its state, follow a path, reject tracking errors, detect and confirm relevant targets, manage mission phases, recover from temporary perception loss, maintain safe stand-off distance and expose its operating state to a human supervisor.

The engineering challenge is therefore one of integration. Guidance, control, computer vision, simulation and operator supervision are often evaluated independently, but an autonomous vessel requires these functions to interact continuously through well-defined interfaces. ROS 2 provides the modular publisher–subscriber, service and action communication model needed for such distributed robotic systems, while Gazebo provides a repeatable environment in which vehicle motion and sensor behavior can be evaluated before hardware trials.

This project addresses the integration problem by building a staged ROS 2–Gazebo autonomous marine research framework. A catamaran-style USV is simulated in Gazebo Harmonic. Natural cubic splines create reference paths, LOS guidance generates heading commands, and PID/SMC controllers drive differential port/starboard thrust. Camera-based target perception is integrated with a mission finite-state machine and depth-derived range. The final architecture also contains cooperative leader–follower control and the MACS (Marine Autonomy Control Station)supervisory GUI.

1.2 Objectives

The principal objective is to develop and validate a modular autonomous marine vehicle research framework that can be demonstrated reliably in simulation while preserving clear interfaces for future hardware integration. The specific objectives are:

- Develop a stable Gazebo Harmonic marine scene and catamaran USV with buoyancy, odometry, IMU and differential-thrust actuation.
- Generate smooth straight, curved and S-shaped reference paths using natural cubic splines and provide ROS 2 guidance outputs.

- Implement LOS path-following guidance and selectable PID/SMC heading control.
- Quantify navigation performance using cross-track error, heading error, path progress and RMSE metrics.
- Integrate YOLO-based maritime target detection with simulated camera data and a thermal-style assigned-temperature validation pipeline.
- Implement temporal confirmation, target-loss recovery, reacquisition and independent depth-derived stand-off arrival.
- Integrate the full search–detect–confirm–acquire–complete mission workflow.
- Extend the framework to two-USV cooperative in-trail formation control with an 8 m desired spacing.
- Develop MACS(Marine Autonomy Control Station) as a real-time supervisory GUI for telemetry, mission state, perception, cooperative control, health and Safe Stop.
- Validate the framework using automated unit/regression tests and short live Gazebo scenarios without overstating simulated capabilities.

1.3 Scope and Limitations

The scope includes simulation, guidance, control, perception, mission autonomy, cooperative operation and supervisory visualization. The system is a research framework rather than a field-qualified autonomous vessel. It provides a repeatable environment for algorithm development and integration testing, with explicit separation between what is simulated and what would require future hardware validation.

- The water surface visual upgrade is a static wavy/PBR mesh; it does not physically generate nonlinear wave forces.
- Buoyancy and hydrodynamic behavior are simulated for repeatable research rather than full high-fidelity computational fluid dynamics.
- Thrusters are modeled as direct force inputs; detailed propeller, motor and cavitation dynamics are outside the current scope.
- The thermal sensing pipeline is assigned-temperature/pseudo-thermal and is not a calibrated LWIR camera model.
- The detector is presented as a generic vessel detector because the class mapping between the available YAML mapping and checkpoint was not fully trustworthy.
- Cooperative control is validated for a two-USV leader–follower scenario rather than a large swarm.
- Physical GPS, real IMU, radio, motor drivers, gimbal hardware and sea trials are future work.

- After completion of some frozen navigation scenarios, a fresh simulation session is required for another path; MACS reports this limitation rather than pretending restart support exists.

CHAPTER 2

LITERATURE SURVEY

2.1 ROS 2 for Autonomous Marine Systems

Modern robotic systems require modular, scalable and reliable communication between sensing, planning, control and supervision components. Macenski et al. describe ROS 2 as a redesigned robotics middleware intended to address production-grade communication, distributed deployment and reliability requirements beyond ROS 1 [1]. These characteristics are directly relevant to autonomous marine systems, where odometry, camera data, control diagnostics and mission state are generated by independent processes operating at different rates.

The present work adopts this modular philosophy. Guidance, control, perception, mission management, cooperative coordination and GUI supervision remain separate ROS 2 nodes/packages, allowing targeted tests and controlled integration rather than a monolithic application.

2.2 Marine Guidance and LOS Path Following

Marine guidance and control literature commonly separates path generation, guidance and low-level control. Fossen's treatment of marine craft guidance, navigation and control provides a standard framework for describing vessel kinematics, hydrodynamics, LOS guidance laws and feedback control [2]. LOS guidance is attractive for underactuated surface craft because it converts geometric path error into a desired course/heading rather than requiring the vessel to exactly track every point on a trajectory.

This project follows the same layered approach: natural cubic splines provide smooth reference geometry; LOS computes the desired heading and speed; a selectable controller then produces yaw-rate/thrust demands. The separation permits the same path and guidance system to be evaluated with different controllers.

2.3 Feedback Control: PID and Sliding Mode Control

PID control is widely used because of its simplicity, interpretability and strong practical baseline performance. The proportional term responds to present error, the integral term to accumulated error

and the derivative term to error variation. In this project the final integral gain is zero, producing a PD-like realization within the PID framework. Sliding Mode Control (SMC) was implemented as a robust nonlinear alternative. SMC drives a selected error surface toward a sliding manifold and can tolerate modeling uncertainty, although aggressive switching can create chattering if not softened through a boundary layer.

The literature motivates evaluating both approaches rather than assuming one is globally superior. Accordingly, this work reports the selected PID and SMC settings and treats the SMC stability evidence as local/conditional to the tested scenario.

2.4 Deep-Learning Maritime Perception

The YOLO family provides single-stage object detection that can localize and classify objects in a single inference pipeline [3]. Maritime studies have applied YOLO variants to optical, infrared and radar imagery to reduce dependence on continuous human observation [4]–[7]. The present work uses a lightweight YOLOv5n6 checkpoint to explore real-time vessel detection within the autonomous mission architecture rather than treating detection as an isolated computer-vision demo.

The integration challenge is important: a detector can provide bounding boxes and confidence, but autonomous behavior additionally requires confirmation logic, target-loss handling, independent range estimation and a mission state machine. The project therefore separates perception confidence from depth-derived range and does not claim that YOLO itself measures distance.

2.5 Multi-USV Cooperative Control

Multi-vehicle marine systems can improve coverage and redundancy by coordinating multiple craft. Leader–follower architectures are a practical starting point because one vehicle can track a global path while another regulates relative position/spacing. The project implements an in-trail cooperative configuration in which the follower uses leader state information and targets an 8 m spacing.

2.6 Human Supervision and Safety

Autonomy does not remove the need for supervisory oversight. A useful operator interface must distinguish required, not-required, stale and invalid data; prevent duplicate mission starts; preserve important final measurements; and provide a safety command independent of normal mission logic. MACS was therefore designed as a supervisory layer that never directly competes for low-level thrust topics. Instead, it consumes authoritative ROS 2 state and uses explicit supervisory services/actions.

2.7 Overall Literature Gap and Project Contribution

Existing work often concentrates on one layer—marine control, perception, ROS middleware or multi-vehicle coordination. The contribution of this project is the integrated research workflow: a Gazebo marine vehicle, spline/LOS navigation, two selectable controllers, vessel perception, mission FSM, depth-based stand-off arrival, cooperative two-USV operation and a state-aware supervisory GUI. The emphasis is not on claiming a new state-of-the-art algorithm in every layer; it is on producing a modular, testable framework in which these layers operate together and their limitations remain visible.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The development and validation were performed on a Lenovo LOQ class laptop with an Intel Core i5-13450HX processor, 16 GB RAM and an NVIDIA RTX 4050 GPU. The primary validation environment used WSL2/WSLg under Windows 11. GPU access was not consistently available to the Linux simulation stack, so the perception throughput figures reported in this work are CPU-oriented. This makes the reported inference performance conservative relative to a properly configured CUDA deployment.

Component	Development requirement / role
CPU	Multi-core x86 processor capable of running Gazebo, ROS 2 and Python nodes concurrently.
Memory	16 GB used during development; 8 GB is a practical minimum for reduced scenarios.
GPU	RTX 4050 available on host; Linux GPU acceleration was not relied upon for the validated CPU perception measurements.
Storage	Sufficient SSD capacity for ROS 2 workspace, Gazebo assets, model weights, logs and evidence.
Display	1920×1080 primary validation; MACS also validated at 1366×768.

3.2 Software Requirements and Architecture

Software	Version / purpose
Ubuntu	22.04.5 LTS
ROS 2	Humble Hawksbill
Gazebo	Harmonic 8.15.0

Software	Version / purpose
Python	3.10
ros_gz_bridge	ROS 2 ↔ Gazebo Transport bridge
PyTorch / YOLOv5	Object detection inference
Qt compatibility layer	PySide6/PyQt5/python_qt_binding support for MACS
RViz2	ROS path, marker and TF visualization

The working ROS 2 workspace is /home/sireesha/marine_ws. The architecture is package-oriented and uses ROS topics for streaming data, services for supervisory commands such as Safe Stop, and an action interface for mission execution. The project intentionally avoided direct GUI publication to /asv/cmd_vel or thruster topics, preserving the control architecture and preventing competing publishers.

3.3 Operational Deployment Considerations

For future physical deployment, the simulation interfaces map naturally to hardware requirements: GPS/INS or fused odometry, IMU, calibrated camera/thermal camera, a range sensor or depth/ranging estimator, motor controllers, network/radio communication and a sealed compute unit. Maritime deployment would additionally require waterproofing, power conditioning, electromagnetic compatibility, fail-safe propulsion disable and sea-trial validation. These are not claimed as completed in this report.

CHAPTER 4

METHODOLOGY / SYSTEM DESIGN

4.1 System Overview

The framework uses a layered architecture in which the environment and vehicle model produce state, guidance converts the reference path into navigation commands, control converts navigation error into thrust, perception supplies target observations, mission logic arbitrates behavior, cooperative control coordinates two vehicles, and MACS provides supervisory visibility and safety commands.

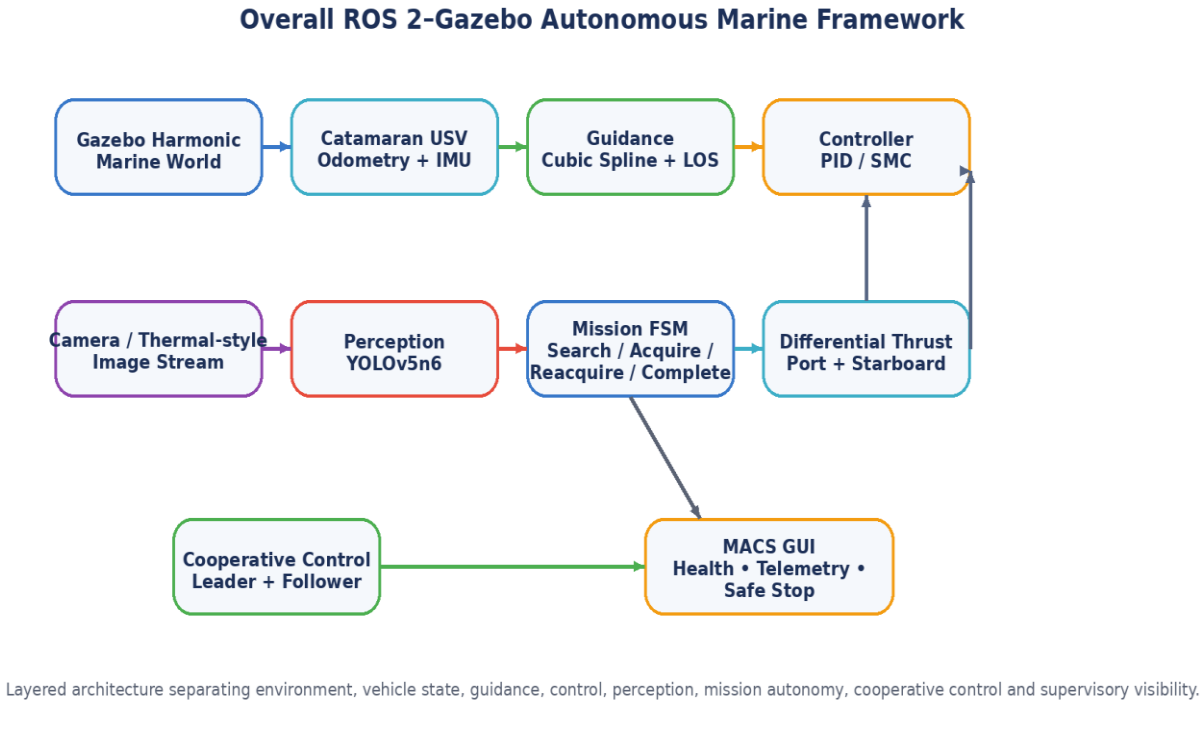


Figure 4.1: Overall ROS 2–Gazebo autonomous marine framework.

4.2 Marine Environment and Catamaran Model

Phase 1 established a calm-water Gazebo Harmonic world, visual water surface and buoyant floating scene. Phase 2 introduced the catamaran USV with differential port/starboard thrust. The final flotation height was approximately 0.112 m in the validated model. The vehicle achieved approximately ±0.79 m/s forward/reverse speed and ±0.39 rad/s turning rate in the baseline characterization.

The ocean appearance was later improved using a static wavy mesh and PBR normal mapping. This was a presentation enhancement only; the physical buoyancy remained a calm-water analytical

simulation. This distinction is important because the visual waves are not used as physical nonlinear disturbances.

4.3 Path Generation with Natural Cubic Splines

Reference paths are generated from waypoints using natural cubic splines. Compared with piecewise straight segments, spline interpolation provides continuous curvature and smoother heading variation, which is more appropriate for a marine vehicle with finite turning capability. Three representative paths were retained for validation: straight, curved and S-shaped.

4.4 LOS Guidance and Cross-Track Error

The LOS guidance layer measures the vehicle's geometric relation to the reference path and generates a desired heading and speed. Cross-track error (CTE) is the perpendicular displacement from the reference path. Path progress, desired heading and CTE are published independently so that controller behavior can be assessed without conflating guidance and actuation.

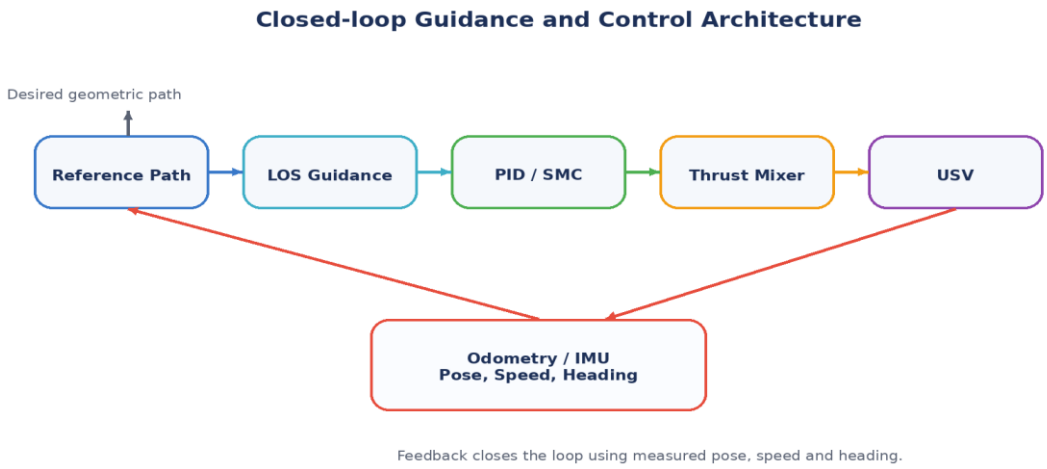


Figure 4.2: Closed-loop path following architecture.

4.5 PID Controller

The PID controller computes a yaw-rate/control demand from heading error. The final selected gains were Kp = 0.5, Ki = 0 and Kd = 0.6. The zero integral gain avoided unnecessary accumulation in the tested simulation, while proportional action corrected heading error and derivative action damped rapid changes. The controller output is mixed into differential port/starboard thrust.

4.6 Sliding Mode Controller

The SMC implementation provides a second selectable control law for research comparison. The selected parameter set was $\lambda = 0.15$, $k = 0.05$, $\varphi = 0.08$ and $\omega = 0.5$. The implementation uses a boundary-layer concept to limit aggressive switching. The report does not claim global SMC superiority over PID; the evaluation is scenario-dependent and the available stability evidence is local/conditional.

4.7 Perception Pipeline

The perception subsystem uses a YOLOv5n6 nano P6 checkpoint with 10 trained classes and approximately 3.11 million parameters, executed with a 640×640 input in the integrated experiments. RGB validation evaluated 273 frames but produced no accepted detections. Rather than hiding this negative result, the framework was extended with a controlled assigned-temperature thermal-style sensing experiment. Live thermal-style trials produced detections suitable for mission integration.

Detection outputs include a bounding box, generic vessel label and confidence. Range is independently derived from the simulated depth channel. This separation prevents the common error of treating YOLO bounding-box output as a physical distance sensor.

4.8 Autonomous Mission State Machine

The mission manager coordinates search, temporal confirmation, acquisition, target-loss recovery, reacquisition and terminal stand-off arrival. A detection must remain sufficiently consistent before it is promoted to a mission target. If the target is lost during the approach, the system enters recovery/reacquisition behavior rather than immediately declaring failure.

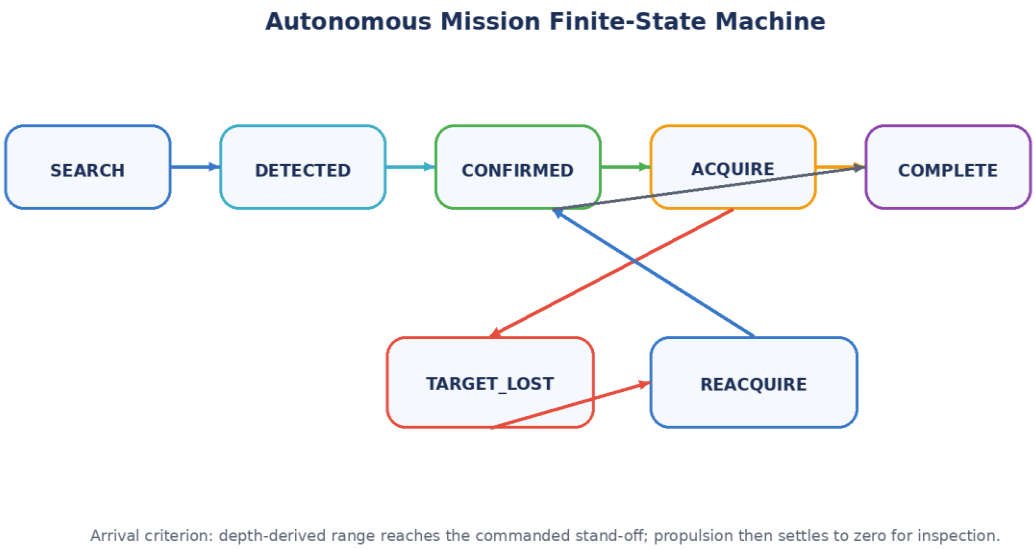


Figure 4.3: Autonomous mission finite-state-machine concept.

4.9 Depth-Based Stand-Off Arrival

The mission uses independent depth-derived range to determine arrival. The final nominal stand-off target is 12.0 m. When the vehicle satisfies the arrival condition, propulsion is brought to zero before inspection/final mission reporting. A successful integrated run terminated at 11.79 m, close to the commanded stand-off while remaining inside the mission acceptance logic.

4.10 Cooperative Leader–Follower Control

The framework was extended to two USVs using independent namespaces and state relays. The leader follows a reference path while the follower regulates its position relative to the leader. The desired formation geometry is an 8 m in-trail spacing. The cooperative coordinator publishes authoritative state used by the GUI and validation scripts.

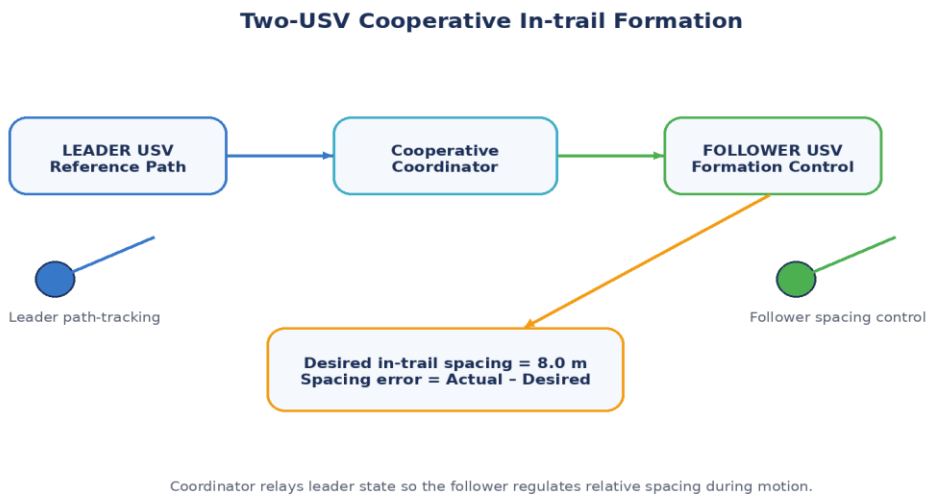


Figure 4.4: Two-USV cooperative in-trail formation concept.

4.11 MACS Supervisory GUI

MACS (Marine Autonomy Control Station) consolidates system health, live telemetry, navigation, perception, cooperative status, research plots and supervisory commands into a single Qt interface. A dedicated ROS bridge thread prevents ROS callbacks from blocking the GUI. The interface contains Overview, Navigation, Perception, Cooperative, Research and Settings screens.

A centralized OperatorState and command-permission model ensures that the top status bar, command panel, mission dialog and safety bar do not independently infer conflicting states. MACS distinguishes FRESH, STALE/LOST, INVALID and NOT REQUIRED stream states, while perception measurements additionally distinguish LIVE, NO CURRENT MEASUREMENT, LAST VALID and FINAL.

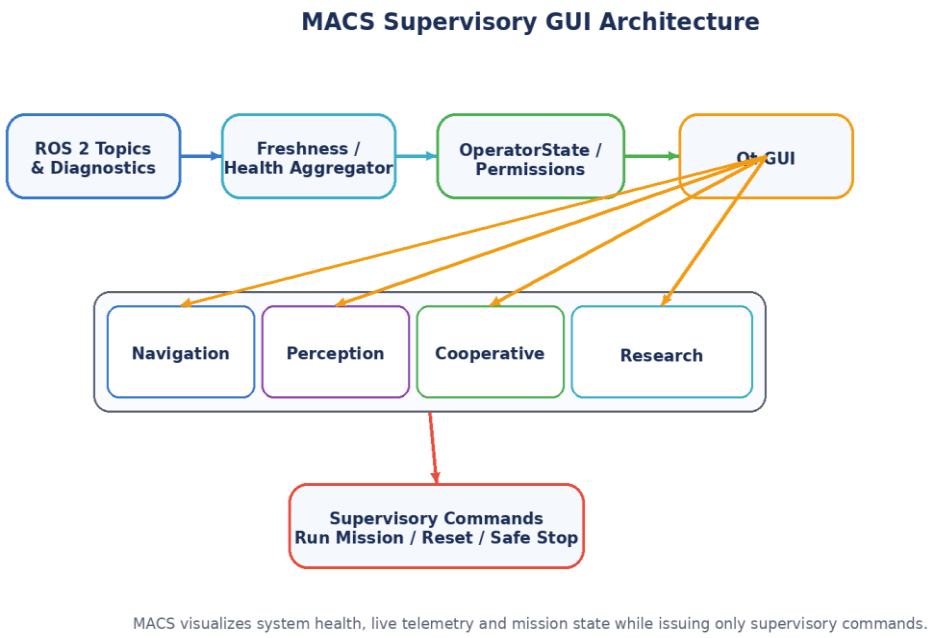


Figure 4.5: MACS supervisory architecture.

4.12 Safety and Safe Stop

Safe Stop is a supervisory safety function that disables active control and verifies zero port/starboard propulsion. The final logic verifies approximately zero thrust (within a small ±0.01 N threshold) with the controller inactive for at least 0.5 s. Safe Stop guarantees zero propulsion, not instantaneous zero vessel velocity; the USV may coast briefly because of inertia.

CHAPTER 5

IMPLEMENTATION

5.1 Development Environment and Workspace

The system was implemented in the ROS 2 workspace /home/sireesha/marine_ws. Ubuntu 22.04.5 LTS, ROS 2 Humble and Gazebo Harmonic were used throughout the final framework. The workspace contains marine description/simulation packages, guidance and control packages, mission/perception packages, cooperative scripts, interface definitions and the marine_gui package. Development was carried out incrementally so that each phase had a known regression baseline before the next layer was added.

5.2 ROS 2 Interfaces

Representative interfaces include /asv/odom for navigation state, /asv/guidance/reference_path and associated guidance diagnostics, /asv/control/diagnostics and active state, /asv/thrusters/port/thrust and /asv/thrusters/starboard/thrust, /asv/perception/annotated_image, /asv/camera/depth, /asv/mission/state and cooperative leader/follower state topics. MACS adds /marine/system_health and supervisory services/actions such as /marine/safe_stop and /marine/run_mission.

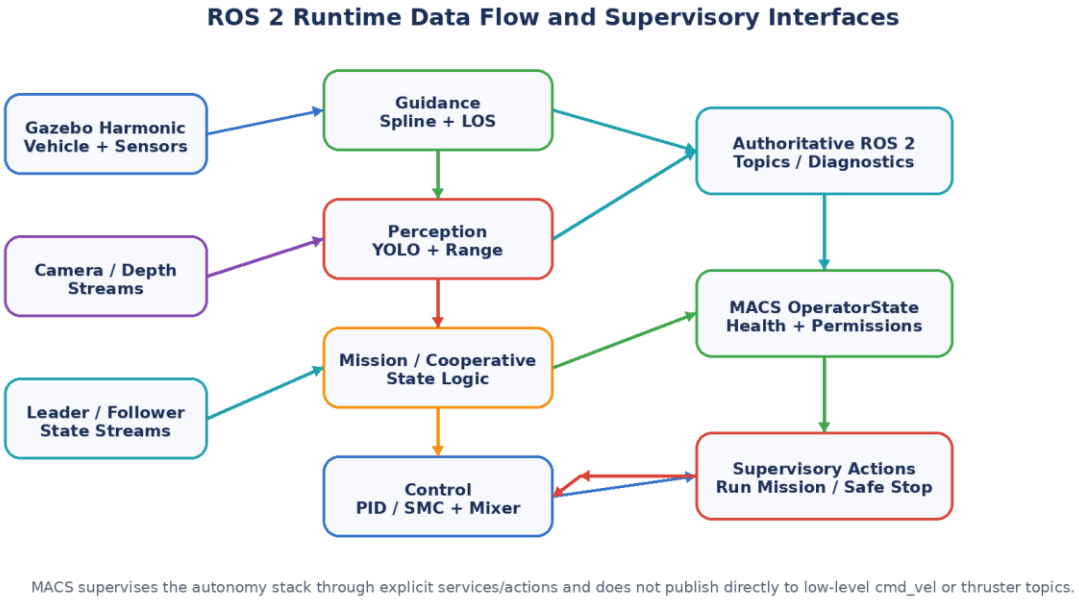


Figure 5.1: ROS 2 runtime data flow and supervisory interface architecture.

5.3 Vehicle State, TF and Cooperative Namespaces

The cooperative phase uses namespaced leader and follower odometry/IMU streams and a vehicle-state compatibility relay. Independent TF subtrees prevent the two vehicles from overwriting each other's frames, while controller-compatible odometry is republished where necessary to preserve the frozen single-USV controller interface.

5.4 Guidance and Controller Integration

Guidance publishes the desired heading, desired speed, CTE and progress. The active controller subscribes to the required navigation state, computes control effort and sends it through the mixer to port/starboard thrust. Controller selection remains explicit so that PID and SMC can be tested with the same path and vehicle setup.

5.5 Perception and Mission Integration

The camera/perception pipeline publishes annotated images and detection information. The mission layer performs temporal confirmation and state transitions. Target bearing is inferred from the image position, while the final distance decision uses depth-derived range. During target loss, missing confidence/range is represented as an unavailable current measurement rather than as a false zero value.

Figure 5.2: End-to-end autonomous mission execution workflow with target-loss recovery.

5.6 MACS Implementation

MACS uses a Qt compatibility layer supporting PySide6, PyQt5 and python_qt_binding. RosBridge runs ROS communication on a separate QThread and emits typed Qt signals into AppState. Custom QPainter widgets render strip charts and navigation displays without heavy plotting dependencies. The GUI remains usable when some subsystems are intentionally not required.

Mission controls are asynchronous. START is gated by connection, health, odometry, controller readiness, safety state and mission lifecycle. A major lifecycle correction removed the circular requirement that guidance must be fresh before navigation can start; guidance instead becomes mandatory after the mission goal is accepted and must appear within a 2 s startup grace period, otherwise GUIDANCE_START_TIMEOUT is reported.

5.7 Demo Runner and Operational Workflow

Four presentation scenarios are exposed through the managed demo runner: basic, pid, mission and cooperative. Each scenario launches the appropriate Gazebo, bridge, guidance/control,

mission/cooperative and visualization components. For final presentation reliability, each major scenario is demonstrated from a fresh simulation and fresh MACS session because the frozen completed-path interfaces do not support arbitrary in-session path replacement.

- ./scripts/run_marine_demo.sh basic --gui
- ./scripts/run_marine_demo.sh pid --gui --rviz
- ./scripts/run_marine_demo.sh mission --gui --rviz
- ./scripts/run_marine_demo.sh cooperative --gui --rviz

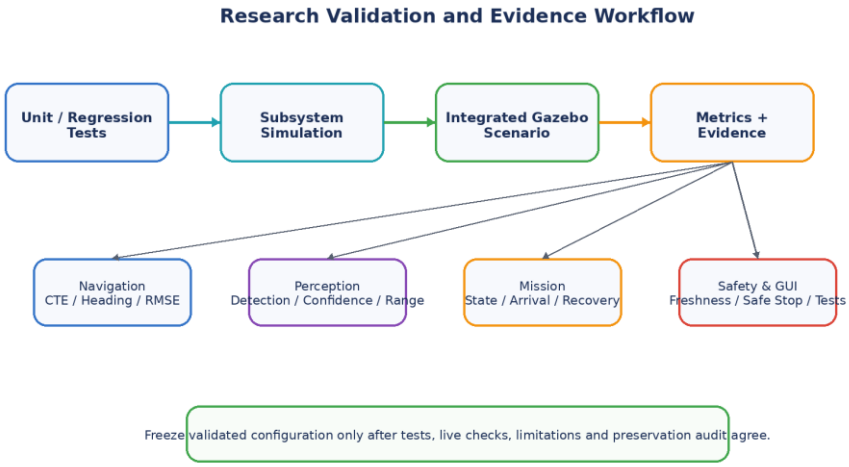


Figure 5.3: Research validation and evidence workflow used before configuration freeze.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Phase-Wise Validation Summary

Phase	Primary result
Phase 1	Water + buoy scene validated; marine world baseline PASS.
Phase 2	Catamaran differential thrust validated; forward/reverse ≈ ±0.79 m/s, turning ≈ ±0.39 rad/s.
Phase 3	Straight/curved/S paths validated; 32 guidance + 4 mixer tests passed; 662 guidance samples matched odometry.
Phase 4	PID LOS controller finalized at Kp=0.5, Ki=0, Kd=0.6; 63 tests passed.
Phase 5B/C	RGB YOLO trial: 273 frames, zero accepted detections; CPU throughput ≈ 4.68–5.05 FPS.
Phase 5D	Thermal-style detection validated; 20/20 live frames detected in the final live check; 96 tests passed.
Phase 5E	Integrated mission: 3/6 full mission trials passed, 1/2 repeat trials passed; successful arrival 11.79 m; 207 tests passed.
Phase 8/9	Cooperative and final research framework frozen; historical regression suite reached 515/515 tests.
MACS final	186 GUI/operator-state tests passed; 1920×1080 and 1366×768 layouts verified.

6.2 Navigation Performance

Reference Path	CTE RMSE	Completion / observation
Straight	0.569 m	Stable tracking in final PID validation.

Reference Path	CTE RMSE	Completion / observation
Curved	0.574 m	Tracking maintained through continuous curvature.
S-shaped	0.583 m	Highest complexity path; error remained close to other scenarios.

The similarity of CTE RMSE across the three reference paths indicates that the tuned PID/LOS combination maintained comparable tracking quality as path complexity increased. The S-shaped path produced the highest RMSE, as expected, but the increase was modest. These values are simulation metrics and should not be interpreted as field-sea accuracy.

Table 6.1: Navigation and Control Evaluation Matrix

Evaluation item	Metric / criterion	Observed result	Interpretation	Status
Straight path	CTE RMSE / completion time	0.569 m / 93.392 s	Stable baseline tracking	PASS
Curved path	CTE RMSE / completion time	0.574 m / 105.912 s	Comparable error under continuous curvature	PASS
S-shaped path	CTE RMSE / completion time	0.583 m / 132.279 s	Tracking maintained under changing curvature	PASS
PID configuration	Selected gains	Kp 0.5, Ki 0, Kd 0.6	Final validated path-following baseline	VERIFIED
SMC configuration	Selected parameters	lambda 0.15, k 0.05, phi 0.08, omega 0.5	Scenario-dependent; local/conditional stability evidence	CONDITIONAL

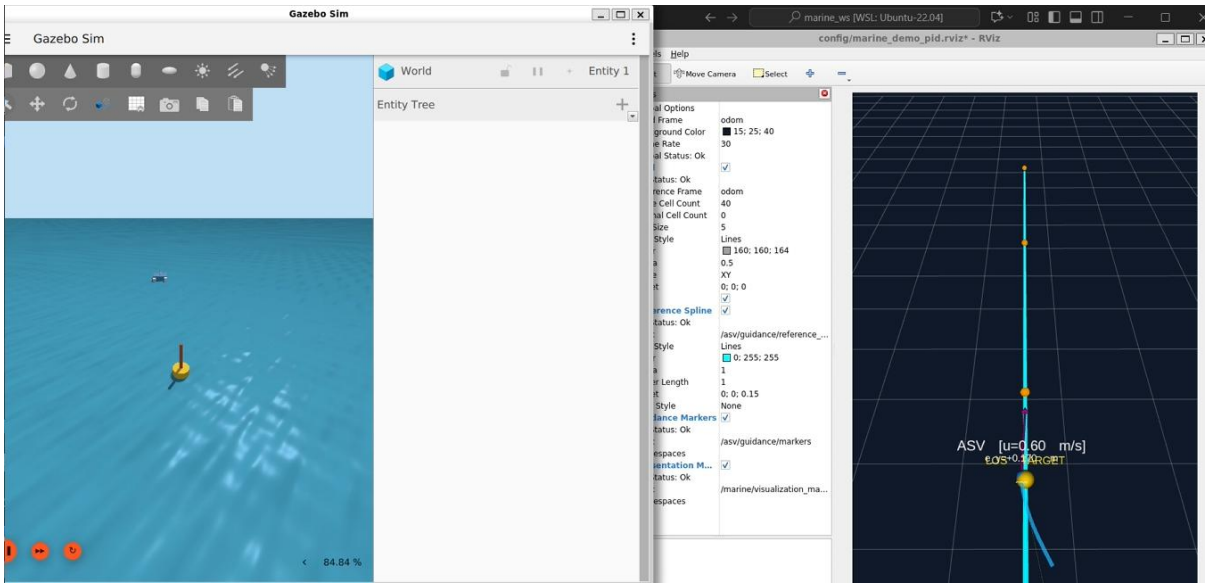


Figure 6.1: Gazebo and RViz visualization of autonomous path following using the reference spline and vehicle guidance markers.

6.3 Perception Results

The RGB experiment is an important negative result: 273 frames were processed with no accepted detections. CPU throughput was approximately 4.68–5.05 FPS. The project therefore did not claim successful RGB target recognition under that configuration. Instead, the controlled thermal-style experiment was used to validate the integration path between sensing, detection and mission logic.

In the assigned-temperature thermal-style pipeline, repeated offline and live checks produced consistent vessel detections. The detector was then coupled to temporal confirmation, search behavior and range-aware mission logic. Because the available checkpoint/YAML class mapping contained a conflict, the final GUI intentionally reports the target generically as VESSEL rather than asserting a potentially incorrect ship subclass.

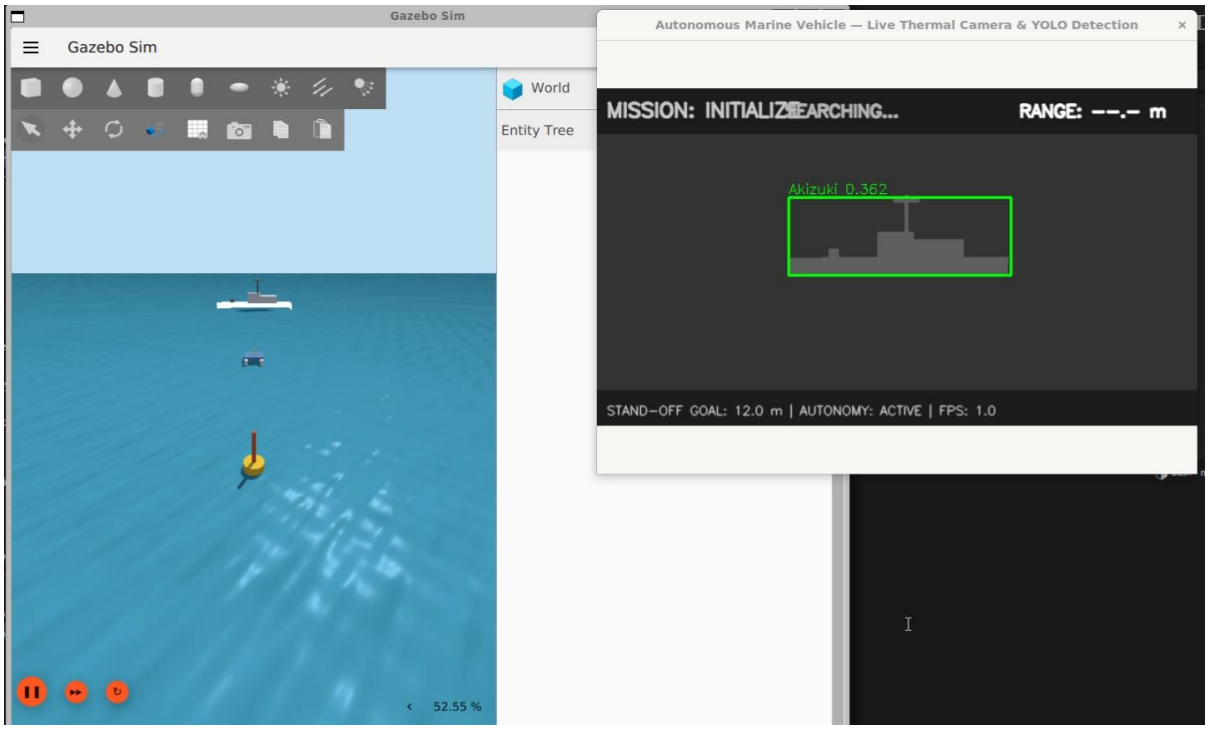


Figure 6.2: Live thermal-style camera output with YOLO vessel detection during the autonomous maritime perception experiment.

6.4 Search and Integrated Mission Results

Two three-lane search validations each took approximately 205.5 s and achieved an overall CTE RMSE of approximately 0.557 m in the thermal-style search validation. The integrated mission subsequently combined search, detection, confirmation, acquisition, target-loss handling, reacquisition and depth-based arrival.

The integrated mission was intentionally evaluated through repeated full runs rather than only unit tests. Three of six full missions passed and one of two repeated validation runs passed. The successful final arrival reached 11.79 m against the nominal 12.0 m stand-off objective. These results demonstrate end-to-end feasibility while also showing that perception loss and integration timing remain meaningful limitations.

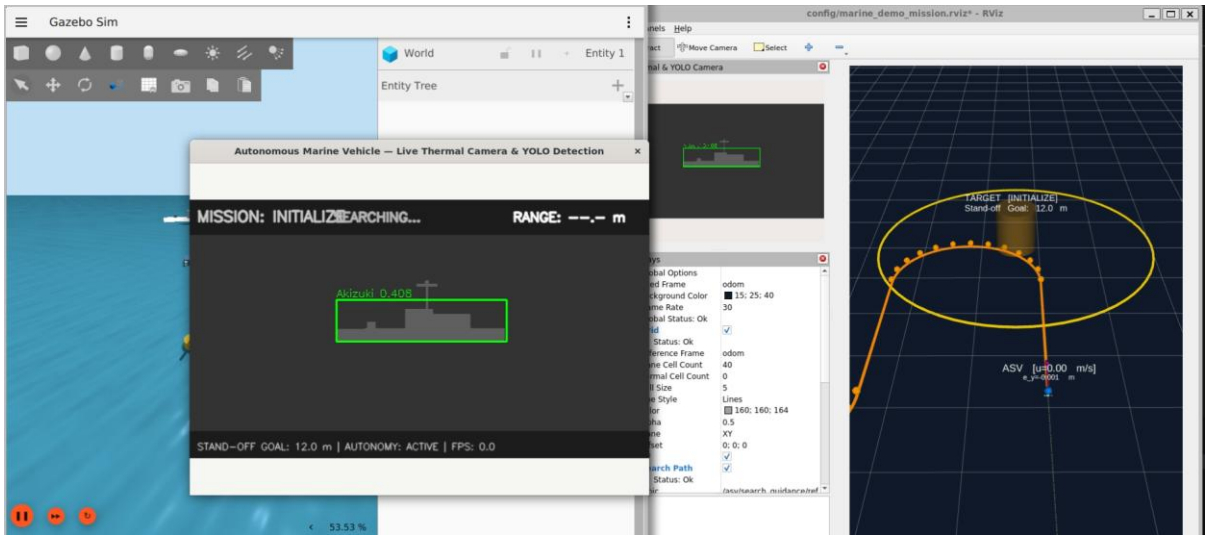


Figure 6.3: Integrated mission demonstration showing Gazebo, thermal-style target detection, search trajectory and stand-off visualization in RViz.

Table 6.2: Perception and Autonomous Mission Evaluation Matrix

Evaluation item	Metric / criterion	Observed result	Interpretation	Status
RGB detector trial	Accepted detections	0 across 273 frames	Negative result retained; motivated thermal-style validation	OBSERVED
CPU inference	Throughput	4.68-5.05 FPS	Baseline perception throughput under CPU-oriented validation	VERIFIED
Thermal-style live detection	Live-frame detection	20 / 20 frames	Consistent simulated target detection for integration	PASS
Three-lane search	Search CTE RMSE	~0.557 m	Search guidance remained within validated tracking behavior	PASS
Integrated mission	Full-run success	3 / 6 missions; 1 / 2 repeat runs	Demonstrated end-to-end autonomy with remaining detection-loss sensitivity	PARTIAL
Stand-off arrival	Final validated range	11.79 m	Close to commanded 12 m nominal stand-off	PASS

6.5 Cooperative Control Results

The cooperative scenario successfully instantiates independent leader and follower vehicles, publishes namespaced vehicle state, and regulates formation toward an 8 m in-trail objective. Formation

acquisition state, actual spacing and spacing error are exposed both in the presentation HUD and through MACS. The final work is limited to two vehicles and does not claim swarm-scale coordination.

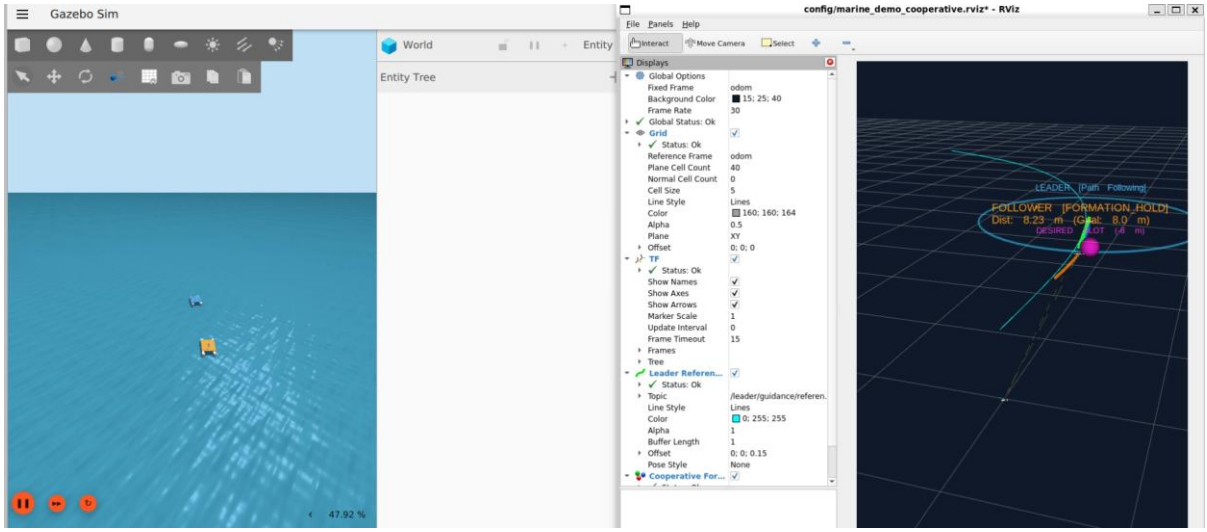


Figure 6.4: Cooperative leader-follower demonstration in Gazebo and RViz showing formation tracking and the in-trail spacing objective.

6.6 MACS Results

MACS was validated not only for visual presentation but also for lifecycle semantics. It correctly locks START during externally launched active missions, distinguishes stream freshness from measurement availability, preserves LAST VALID/FINAL range values, labels expected controller holds separately from faults, and reports unsupported completed-session restarts. The final operator-state consistency suite passed 186 tests. Layout checks passed at both 1920×1080 and 1366×768 without operator-label clipping, overlap or horizontal overflow.

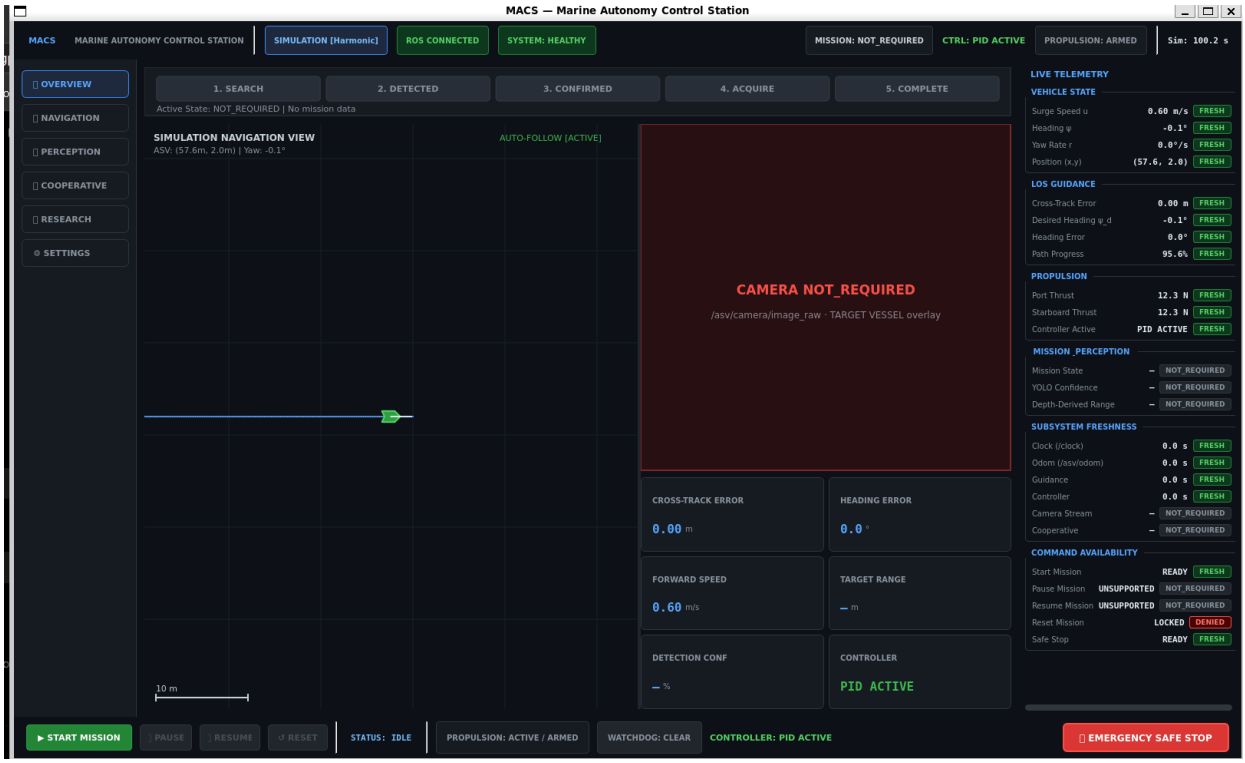


Figure 6.5: MACS PID/navigation view during an active path-following run.

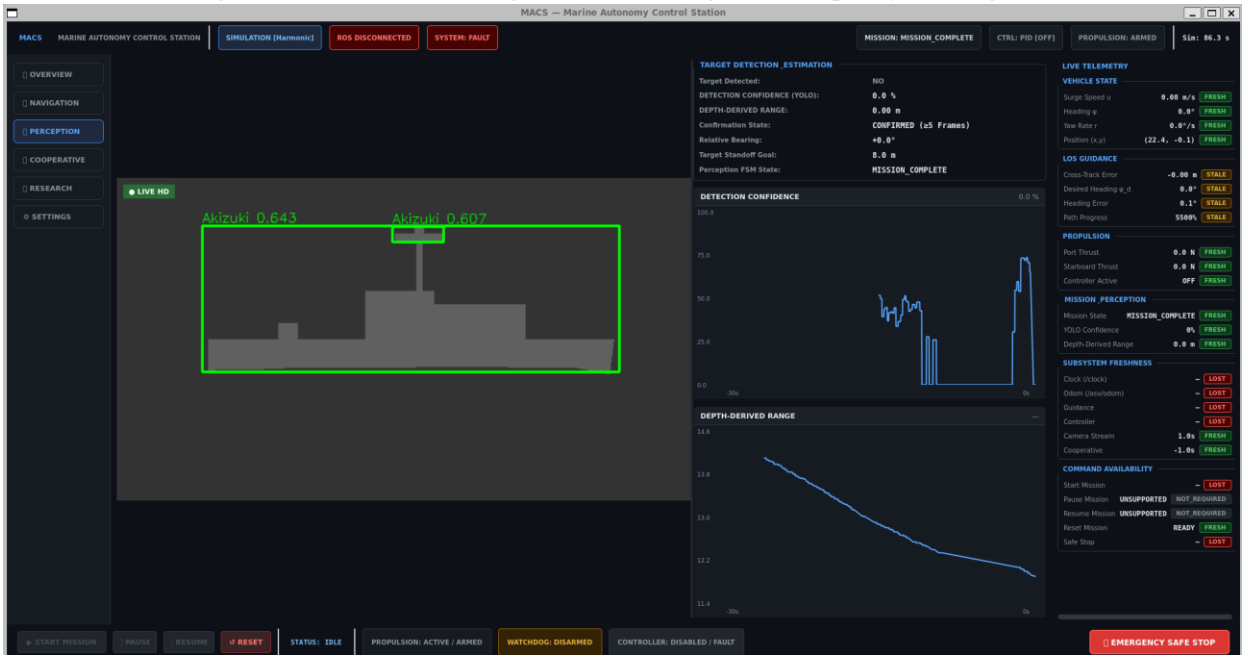


Figure 6.6: MACS perception view showing vessel detection and depth-derived range trend.

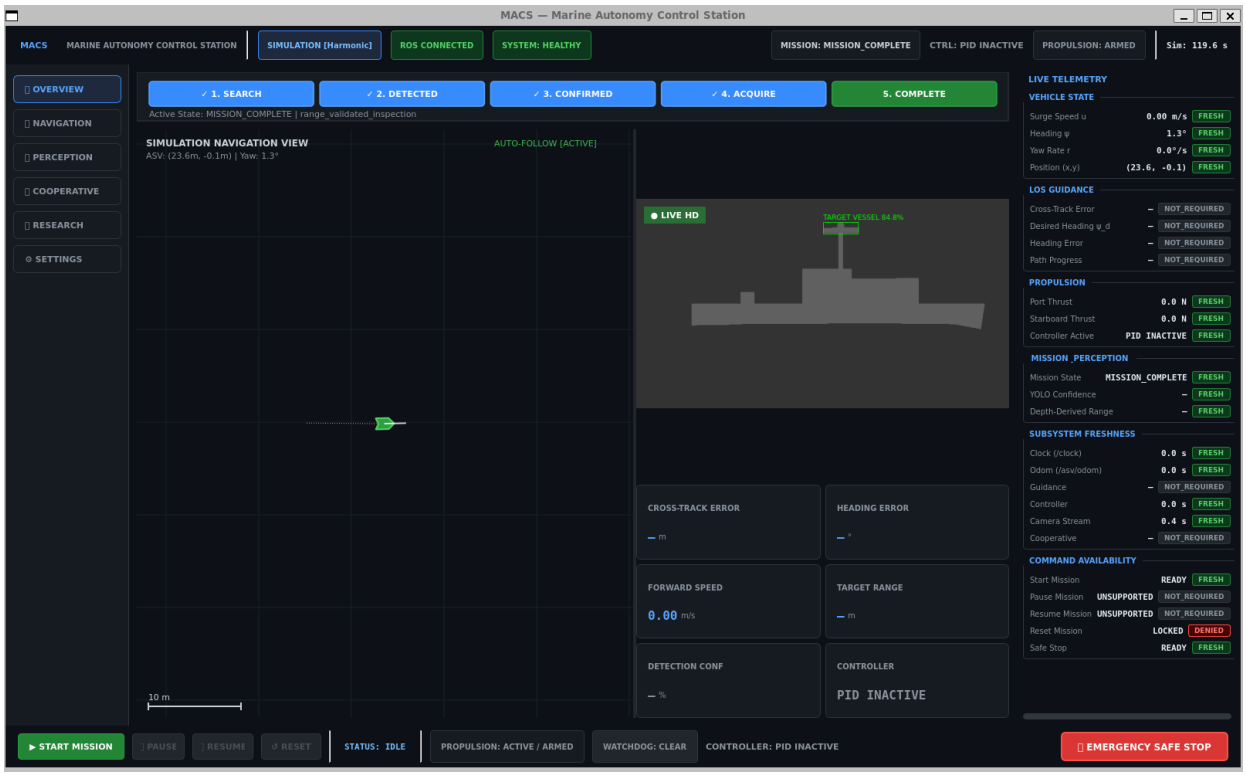


Figure 6.7: MACS mission-complete view after integrated perception/mission operation.

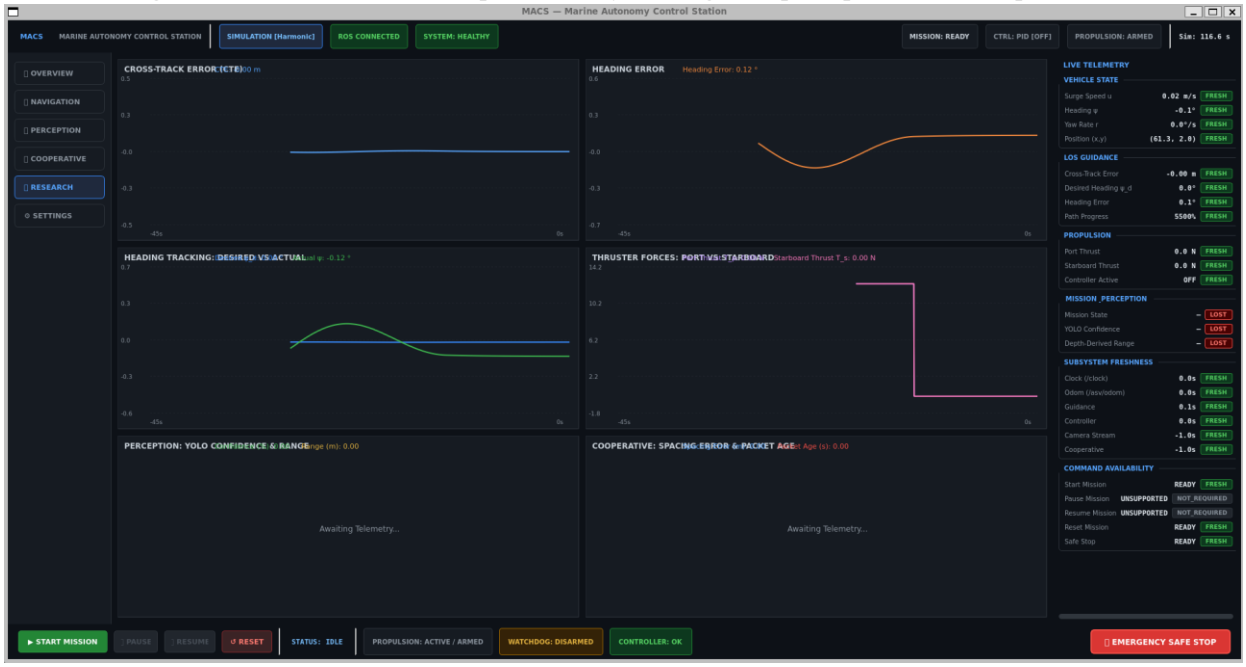


Figure 6.8: MACS research view showing live control and propulsion telemetry.

6.7 Safety and Preservation Validation

Safe Stop verification confirms zero commanded propulsion rather than instantaneous zero physical speed. The controller is disabled and both thruster channels must remain within the zero threshold for

the required verification duration. During the final GUI consistency work, preservation audits confirmed that 19,391 protected files in the frozen autonomy/evidence set remained unchanged, demonstrating that operator-layer corrections did not silently modify the validated core research packages.

Table 6.3: System-Level Verification and Evidence Matrix

Subsystem	Evaluation criterion	Evidence / outcome	Status
Simulation foundation	Buoyancy / vehicle / world launch	Stable marine scene and catamaran operation	PASS
Guidance & control	Spline + LOS + PID/SMC interfaces	Path-following and selectable-control integration validated	PASS
Perception	Detector + depth-derived range	Thermal-style vessel detection integrated; generic vessel label retained	PASS
Autonomous mission	Search / confirmation / acquire / recovery / arrival	End-to-end mission behavior demonstrated with documented limitations	PASS
Cooperative autonomy	Two-USV in-trail objective	Leader/follower framework validated at 8 m desired spacing	PASS
MACS GUI	Regression / operator-state semantics	186 tests passed; lifecycle and command consistency verified	PASS
Safety	Safe Stop verification	Zero propulsion verified with controller inactive over required interval	PASS
Preservation audit	Protected project files	19,391 protected files unchanged during final GUI fixes	PASS

The evaluation matrices consolidate quantitative metrics and integration evidence so that subsystem performance, end-to-end mission behavior, and safety verification can be reviewed independently without overstating the limits of the simulated environment.

6.8 Discussion

The most important result is the successful integration of layers that are commonly demonstrated separately. The framework provides a coherent path from simulated vehicle state to guidance, controller, thrusters, perception, mission decisions, cooperative behavior and supervisory visualization. At the same time, the project records unsuccessful RGB detection trials, partial full-mission success

rates and known restart/physics limitations. This makes the framework suitable as a research baseline rather than an overstated “finished autonomous boat”.

CHAPTER 7

ADVANTAGES AND FUTURE EXTENDABILITY

7.1 Modular ROS 2 Architecture

Navigation, control, perception, mission, cooperative coordination and GUI supervision are separated into independent ROS 2 modules. This improves testability, allows individual subsystems to be replaced, and reduces the risk that one experimental change alters the entire framework.

7.2 Repeatable Simulation-Based Research

Gazebo provides a controlled environment in which the same path, controller gains, target placement and mission sequence can be replayed. Deterministic disturbance experiments further support repeatable controller comparisons, although visual wave motion itself is not used as a physical disturbance source.

7.3 Multi-Layer Validation

The project combines unit tests, integration tests, ROS fixtures, short live Gazebo checks and phase-level mission trials. This layered validation approach is especially valuable in robotics because a component can pass in isolation while failing when timing, topics and lifecycle transitions interact.

7.4 Transparent Limitations

The architecture explicitly represents NOT REQUIRED, INVALID, NO CURRENT MEASUREMENT and unsupported restart conditions instead of hiding them. Similarly, the report documents the RGB detection failure and partial mission pass rate. This transparency makes future improvements measurable.

7.5 Extendability

The interfaces support future replacement of simulated odometry with GPS/INS, simulated camera with physical RGB/LWIR sensing, direct-force thrusters with motor drivers, and two-USV coordination with larger formations. A real gimbal can be integrated through a driver node publishing camera/gimbal state and accepting pan/tilt commands while maintaining the appropriate TF chain.

CHAPTER 8

APPLICATIONS

1. Autonomous maritime surveillance research: repeatable evaluation of search, target confirmation and stand-off approach behaviors.
2. Guidance and controller benchmarking: comparison of PID, SMC and future control laws on common paths and disturbance profiles.
3. Harbour and coastal monitoring research: integration of camera-based target detection with autonomous surface vehicles.
4. Search and inspection missions: autonomous search, target reacquisition and safe stand-off inspection workflows.
5. Cooperative surface robotics: leader–follower formation control, communication-degradation studies and future multi-agent experiments.
6. Human–autonomy supervision research: evaluation of operator-state presentation, command gating, health aggregation and emergency stop behavior.
7. Education and training: demonstration of ROS 2 publishers/subscribers, actions/services, TF, Gazebo, feedback control, YOLO integration and GUI development in a single project.
8. Hardware-in-the-loop preparation: future substitution of simulated sensors and actuators with real interfaces while retaining the ROS 2 architecture.

CHAPTER 9

CONCLUSION

This project successfully developed an integrated autonomous marine Software ROS 2 Humble and Gazebo Harmonic. The work progressed from a basic buoyant marine scene to a catamaran USV with differential thrust, natural cubic-spline paths, LOS guidance, PID/SMC control, vessel perception, autonomous search/approach logic, cooperative two-USV operation and a dedicated supervisory GUI.

The final PID configuration achieved CTE RMSE values near 0.57–0.58 m across straight, curved and S-shaped reference paths. The perception work demonstrated both the limitations of the available RGB detector configuration and the feasibility of integrating a controlled thermal-style detector with the mission framework. The integrated mission reached a successful 11.79 m stand-off arrival in its final successful run, and the cooperative framework established an 8 m leader–follower objective.

MACS completed the system by providing a consistent operator view of vehicle state, guidance, propulsion, perception, mission, cooperative status and safety. Its final 186-test suite and multi-resolution layout validation supported presentation readiness. The system also includes a verified Safe Stop concept that ensures propulsion is disabled before inspection or emergency handling.

The project should be understood as a validated simulation and software research platform rather than a fully field-qualified autonomous vessel. Its main achievement is the integration and disciplined validation of multiple autonomy layers under a common ROS 2 architecture, providing a strong foundation for hardware integration, improved perception and higher-fidelity marine dynamics in future work.

CHAPTER 10

FUTURE SCOPE

10.1 Real Hardware Integration

Future work should connect the ROS 2 framework to a physical USV with GNSS/GPS, IMU, motor controllers, power monitoring and a safety-rated propulsion-disable interface. Hardware-in-the-loop testing should precede open-water trials.

10.2 Calibrated RGB/Thermal Perception

The pseudo-thermal simulation should be replaced or complemented by calibrated RGB and LWIR cameras. A new maritime dataset with verified class labels should be used to train/validate the detector so that generic VESSEL labels can be replaced by trustworthy class-specific predictions.

10.3 Target Tracking and Gimbal Control

Persistent target IDs can be added using a tracking algorithm such as ByteTrack or Deep SORT. A real pan–tilt gimbal can then be driven by the target bearing while publishing gimbal joint state and TF transforms so that camera orientation remains geometrically consistent with navigation.

10.4 Higher-Fidelity Hydrodynamics

The present visual wave mesh should be separated from future physical wave/current models. Higher-fidelity hydrodynamics, wind/current disturbances, actuator lag and propeller characteristics would enable more realistic controller robustness studies.

10.5 Collision Avoidance and COLREG-Aware Planning

The current work focuses on reference-path following and target missions. Future autonomy can incorporate obstacle detection, closest-point-of-approach reasoning and COLREG-aware collision avoidance so that path generation responds to dynamic traffic.

10.6 Larger Cooperative Teams

The two-USV leader–follower framework can be extended to multiple followers, formation reconfiguration, consensus-based coordination and communication-delay/loss studies. This should be validated progressively to avoid conflating network failure with controller failure.

10.7 Formal Safety and Mission Recovery

Future work can add explicit safety monitors, formal fault trees, watchdog recovery policies and deterministic restart/reset interfaces. The present MACS correctly reports unsupported in-session restart conditions; future architecture can add real reset contracts rather than hiding the limitation in the operator layer.

REFERENCES

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” Science Robotics, vol. 7, no. 66, eabm6074, 2022.

[2] T. I. Fossen, Handbook of Marine Craft Hydrodynamics and Motion Control, 2nd ed. Wiley, 2021.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proc. CVPR, 2016, pp. 779–788.

[4] K. Lan, Z. Song, and W. Wang, “Video image naval vessels and ships detection method based on YOLOv5x,” Proc. ACCTCS, 2023, pp. 744–748.

[5] L. Chang, Y.-T. Chen, J.-H. Wang, and Y.-L. Chang, “Modified YOLOv3 for ship detection with visible and infrared images,” Electronics, vol. 11, no. 5, p. 739, 2022.

[6] X. Zhang, L. Zhang, and Y. Wang, “Ship detection in marine radar images based on a modified YOLOv3-tiny,” Proc. IGARSS, 2021, pp. 1230–1233.

[7] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” Proc. CVPR, 2023, pp. 7464–7475.

[8] Ultralytics, “YOLOv5,” GitHub repository and model documentation, https://github.com/ultralytics/yolov5.

[9] Open Robotics, “ROS 2 Humble Documentation,” https://docs.ros.org/en/humble/.

[10] Open Robotics, “Gazebo Harmonic Documentation,” Gazebo Sim documentation.

[11] M. F. Reis et al., “A ROS implementation of the situational awareness and maneuvering systems for an autonomous marine vessel,” Proc. IEEE ICRA, 2019.

[12] N. O’Mahony et al., “Deep learning vs. traditional computer vision,” Advances in Intelligent Systems and Computing, Springer, 2020, pp. 128–144.

[13] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” IEEE TPAMI, vol. 39, no. 6, pp. 1137–1149, 2017.

[14] W. Liu et al., “SSD: Single Shot MultiBox Detector,” Proc. ECCV, 2016, pp. 21–37.

[15] H. Li, J. Tan, and Z. Chen, “Drone-ROS2 ground terminal collaborative detection system based on YOLO target recognition,” IEEE Access, vol. 11, pp. 45210–45220, 2023.